Shape and Color Sorter

A Classical Computer Vision Application Using Python and OpenCV

CSC 126 Computer Vision · Midterm Project · Caraga State University, CCIS

Members: [Name 1], [Name 2], [Name 3] | Repository: [GitHub link]

1. Title and Objectives

Shape and Color Sorter is a Python and OpenCV program that finds colored objects in a scene, identifies each one by color and shape, and counts them. The objectives are to:

- detect objects of six colors (red, orange, yellow, green, blue, purple) and five shapes (triangle, square, rectangle, pentagon, circle);
- process a static image, a recorded video, and a live webcam feed with the same pipeline;
- visualize results with contours, bounding boxes, labels, centroids, masks, and a running count; and
- measure accuracy against labelled test images instead of judging it by eye.

2. Problem and Use-Case

Sorting stations in small factories, recycling lines, and classrooms need to know what each item is before it can be routed. Doing this by hand is slow and error-prone. This project builds the vision step of such a station: a camera looks at a table or conveyor of colored items, and the program reports what each item is and how many of each type are present. The same approach applies to sorting colored parts, counting game pieces, or checking that a kit has the right components.

3. OpenCV Techniques Used

The project uses three of the six laboratory topics. Every frame passes through the same five steps: preprocess, segment, find contours, classify, and visualize.

Topic	Techniques	Why it is needed
(a) Image Operations	cv2.resize to 800 px width; GaussianBlur 5x5; cvtColor BGR to HSV; drawContours, rectangle, putText, circle; addWeighted overlay	A fixed size keeps area thresholds consistent. Blur removes noise and paper texture. Drawing makes the result readable.
(b) Image Segmentation	cv2.inRange per color in HSV space (red uses two hue ranges); morphological opening and closing with a 5x5 elliptical kernel	HSV separates color (hue) from brightness, so shadows don't change the color. Morphology removes specks and fills holes.
(d) Image Analysis and Transformation	Canny edges; findContours (external); approxPolyDP at 4% of perimeter; minAreaRect; image moments; circularity $4\pi A/P^2$	Counting corners identifies polygons. The rotated rectangle makes square vs. rectangle independent of tilt. Circularity identifies circles.

Classification rule. After approxPolyDP, 3 corners is a triangle and 5 is a pentagon. Four corners is a square if the rotated bounding box has an aspect ratio of 1.15 or less, otherwise a rectangle. Any other outline with circularity of 0.80 or more is a circle (a perfect circle scores 1.0). Contours smaller than 600 px^2 are discarded as noise.

4. Results

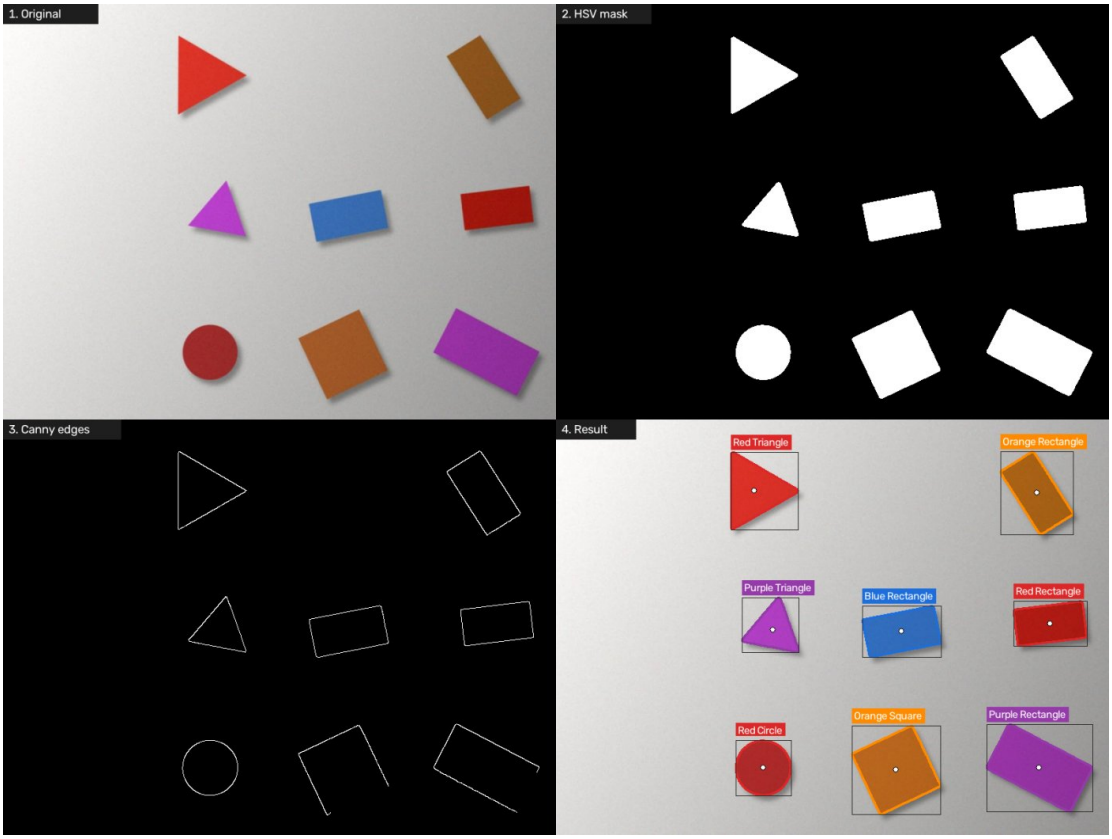


Figure 1. The pipeline on a photo-like test image with shadows and uneven lighting: (1) original, (2) combined HSV mask after morphology, (3) Canny edges, (4) final labelled result. All 8 objects are correct, including tilted squares and rectangles.

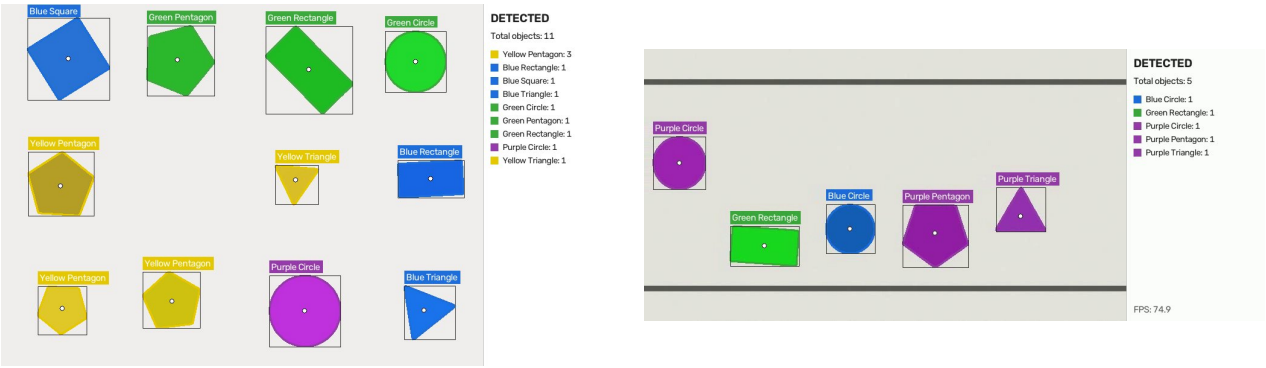


Figure 2. Left: static image with 11 objects, all correctly labelled. Right: a frame from the recorded conveyor video with the count panel and live FPS (about 73 FPS on a laptop CPU).

5. Accuracy, Behavior, and Limitations

Accuracy was measured with **evaluate.py** on 30 generated images (307 objects) whose true color, shape, and position are stored in **ground_truth.json**. Each true object is matched to the nearest detection within 40 px. Three difficulty levels were tested: *clean* (even lighting), *hard* (drop shadows, uneven lighting, noise, blur, JPEG compression), and *stress* (small, faded, dim shapes with heavy noise), designed to find where the method fails.

Test set	Objects	Recall	Precision	Color correct	Shape correct
Clean	100	100%	100%	100%	100%
Hard	97	100%	100%	100%	100%
Stress	110	83.6%	100%	100%	98.9%
All	307	94.1%	100%	100%	99.7%

Behavior. The program never reported an object that wasn't there (0 false positives) and every object it found had the right color. Shape was wrong only once (a small, noisy square read as a pentagon). Rotation did not affect results because of minAreaRect and corner counting.

Limitations. All 18 misses were in the stress set, from two causes found by inspecting each missed object:

- **Faded or dark objects (12 misses).** Their saturation fell below 90 or brightness below 50 to 70, so inRange rejected them. For the same reason, white, black, and grey objects cannot be detected at all.
- **Very small objects (6 misses).** Small triangles produced masks under the 600 px^2 noise filter.
- **Other cases.** Touching shapes of the same color merge into one contour, objects cut off by the frame edge may be misclassified, and colored lighting shifts hues. The included hsv_tuner.py tool recalibrates the ranges for a new room.

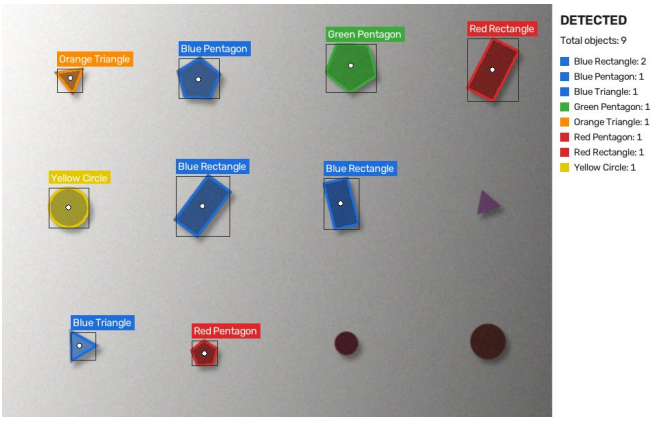


Figure 3. A stress image. Faded and dark shapes on the right side are missed, while the detections that are made are all correct.

6. Conclusion

The Shape and Color Sorter meets its objectives. Using only classical techniques, it labels color with 100% accuracy and shape with 99.7% accuracy across 307 test objects, runs in real time, and works on images, video, and webcam input.

Key learnings. HSV is far more reliable than RGB for color under changing light. Morphology is essential: without it, noise creates false contours. Simple geometric measures (corner count, aspect ratio, circularity) are enough for clean shapes, and measuring accuracy against ground truth revealed failure causes that visual checks would have missed.

Possible improvements. Adaptive or auto-calibrated color thresholds; watershed or distance-transform segmentation to separate touching objects; object tracking with IDs so each item on a conveyor is counted once; and support for more shapes such as hexagons and stars.